

Deployment Guide

Overview

This guide provides step-by-step instructions for deploying the Vidyarthi Book Depot monolithic platform to various environments.

Prerequisites

Required Software

- **Java:** JDK 17 or higher
- **MySQL:** 8.0 or higher
- **Apache Kafka:** 3.5.x or higher
- **Maven/Gradle:** For building the application
- **Docker** (Optional): For containerized deployment

System Requirements

Development Environment:

- CPU: 2 cores
- RAM: 4 GB
- Disk: 20 GB

Production Environment:

- CPU: 4+ cores
- RAM: 8+ GB
- Disk: 100+ GB (with growth)
- Network: High-speed internet connection

Environment Setup

1. Database Setup

Install MySQL

Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install mysql-server
sudo systemctl start mysql
sudo systemctl enable mysql
```

Windows:

Download and install MySQL from <https://dev.mysql.com/downloads/>

Create Database

```
CREATE DATABASE vidyarthi_book_depot
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

CREATE USER 'vbd_user'@'localhost' IDENTIFIED BY 'secure_password';
GRANT ALL PRIVILEGES ON vidyarthi_book_depot.* TO 'vbd_user'@'localhost';
FLUSH PRIVILEGES;
```

Run Database Migrations

```
# Using Flyway or Liquibase
mvn flyway:migrate
# OR
./gradlew flywayMigrate
```

2. Kafka Setup

Install Kafka

Linux:

```
wget https://downloads.apache.org/kafka/3.5.0/kafka_2.13-3.5.0.tgz
tar -xzf kafka_2.13-3.5.0.tgz
cd kafka_2.13-3.5.0
```

Windows:

Download from <https://kafka.apache.org/downloads>

Start Zookeeper

```
bin/zookeeper-server-start.sh config/zookeeper.properties
```

Start Kafka Broker

```
bin/kafka-server-start.sh config/server.properties
```

Create Kafka Topics

```
bin/kafka-topics.sh --create --topic chat-messages --bootstrap-server localhost:9092 --partitions 1
```

```
bin/kafka-topics.sh --create --topic chat-notifications --bootstrap-server localhost:9092 --partitions 1
```

```
bin/kafka-topics.sh --create --topic inquiry-notifications --bootstrap-server localhost:9092 --partitions 1
```

3. Application Configuration

Application Properties

Create application.yml :

```
spring:
  application:
    name: vidyarthi-book-depot

datasource:
  url: jdbc:mysql://localhost:3306/vidyarthi_book_depot
  username: vbd_user
  password: ${DB_PASSWORD}
  driver-class-name: com.mysql.cj.jdbc.Driver
  hikari:
    maximum-pool-size: 10
    minimum-idle: 5
    connection-timeout: 30000

jpa:
  hibernate:
    ddl-auto: validate
  show-sql: false
  properties:
    hibernate:
      dialect: org.hibernate.dialect.MySQL8Dialect
      format_sql: true

kafka:
  bootstrap-servers: localhost:9092
  producer:
    key-serializer: org.apache.kafka.common.serialization.StringSerializer
    value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
  consumer:
    group-id: chat-consumer-group
    key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
    value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
    properties:
      spring.json.trusted.packages: "*"

mail:
  host: smtp.gmail.com
  port: 587
  username: ${EMAIL_USERNAME}
  password: ${EMAIL_PASSWORD}
  properties:
    mail:
      smtp:
```

```
    auth: true
    starttls:
      enable: true

jwt:
  secret: ${JWT_SECRET}
  expiration: 3600000

server:
  port: 8080
  servlet:
    context-path: /
```

Environment Variables

Create `.env` file`:

```
DB_PASSWORD=secure_password
JWT_SECRET=your-secret-key-min-256-bits
EMAIL_USERNAME=your-email@gmail.com
EMAIL_PASSWORD=your-app-password
```

4. Build Application

Using Maven

```
mvn clean package -DskipTests
```

Using Gradle

```
./gradlew clean build -x test
```

The build will create a JAR file in `target/` or `build/libs/` directory.

Deployment Options

Option 1: Standalone JAR Deployment

Run Application

```
java -jar target/vidyarthi-book-depot-1.0.0.jar
```

Run as Service (Linux)

Create systemd service file `/etc/systemd/system/vbd.service` :

```
[Unit]
Description=Vidyarthi Book Depot Application
After=network.target mysql.service kafka.service

[Service]
Type=simple
User=vbd
WorkingDirectory=/opt/vbd
ExecStart=/usr/bin/java -jar /opt/vbd/vidyarthi-book-depot-1.0.0.jar
Restart=always
RestartSec=10
Environment="JAVA_OPTS=-Xmx2g -Xms1g"

[Install]
WantedBy=multi-user.target
```

Enable and start service:

```
sudo systemctl enable vbd
sudo systemctl start vbd
sudo systemctl status vbd
```

Option 2: Docker Deployment

Create Dockerfile

```
FROM openjdk:17-jdk-slim

WORKDIR /app

COPY target/vidyarthi-book-depot-1.0.0.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Build Docker Image

```
docker build -t vidyarthi-book-depot:1.0.0 .
```

Run Docker Container

```
docker run -d \
  --name vbd-app \
  -p 8080:8080 \
  -e DB_PASSWORD=secure_password \
  -e JWT_SECRET=your-secret-key \
  -e EMAIL_USERNAME=your-email@gmail.com \
  -e EMAIL_PASSWORD=your-app-password \
  --network vbd-network \
  vidyarthi-book-depot:1.0.0
```

Docker Compose

Create docker-compose.yml :

version: '3.8'

services:

mysql:

image: mysql:8.0

container_name: vbd-mysql

environment:

MYSQL_DATABASE: vidyarthi_book_depot

MYSQL_USER: vbd_user

MYSQL_PASSWORD: secure_password

MYSQL_ROOT_PASSWORD: root_password

ports:

- "3306:3306"

volumes:

- mysql-data:/var/lib/mysql

networks:

- vbd-network

zookeeper:

image: confluentinc/cp-zookeeper:latest

container_name: vbd-zookeeper

environment:

ZOOKEEPER_CLIENT_PORT: 2181

networks:

- vbd-network

kafka:

image: confluentinc/cp-kafka:latest

container_name: vbd-kafka

depends_on:

- zookeeper

environment:

KAFKA_BROKER_ID: 1

KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181

KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092

KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

ports:

- "9092:9092"

networks:

- vbd-network

app:

build: .


```
container_name: vbd-app
depends_on:
  - mysql
  - kafka
environment:
  SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/vidyarthi_book_depot
  SPRING_DATASOURCE_USERNAME: vbd_user
  SPRING_DATASOURCE_PASSWORD: secure_password
  KAFKA_BOOTSTRAP_SERVERS: kafka:9092
  JWT_SECRET: your-secret-key
  EMAIL_USERNAME: your-email@gmail.com
  EMAIL_PASSWORD: your-app-password
ports:
  - "8080:8080"
networks:
  - vbd-network
```

volumes:

mysql-data:

networks:

vbd-network:

driver: bridge

Start with Docker Compose:

```
docker-compose up -d
```

Option 3: Cloud Deployment

AWS Deployment

Using Elastic Beanstalk:

1. Create Elastic Beanstalk application
2. Upload JAR file
3. Configure environment variables
4. Set up RDS MySQL instance
5. Set up MSK (Managed Streaming for Kafka)
6. Deploy application

Using EC2:

1. Launch EC2 instance
2. Install Java, MySQL, Kafka
3. Upload application JAR
4. Configure security groups
5. Set up RDS for MySQL
6. Run application

Azure Deployment

1. Create App Service
2. Configure MySQL database
3. Set up Event Hubs (Kafka alternative)
4. Deploy application
5. Configure environment variables

Production Checklist

Security

- ☐ Change default database passwords
- ☐ Use strong JWT secret (min 256 bits)
- ☐ Enable HTTPS/SSL
- ☐ Configure firewall rules
- ☐ Set up rate limiting
- ☐ Enable CORS for specific origins only
- ☐ Use environment variables for secrets
- ☐ Enable database encryption
- ☐ Set up backup encryption

Performance

- ☐ Configure connection pool size
- ☐ Enable database query caching
- ☐ Set up application-level caching
- ☐ Configure Kafka consumer groups
- ☐ Set up load balancing (if multiple instances)
- ☐ Enable Gzip compression

- ☐ Configure CDN for static assets

Monitoring

- ☐ Set up application logging
- ☐ Configure log aggregation
- ☐ Set up health check endpoints
- ☐ Configure metrics collection
- ☐ Set up alerting
- ☐ Monitor database performance
- ☐ Monitor Kafka lag
- ☐ Set up error tracking

Backup

- ☐ Configure database backups
- ☐ Set up automated backup schedule
- ☐ Test backup restoration
- ☐ Configure backup retention policy
- ☐ Set up off-site backup storage

Health Checks

Application Health

```
curl http://localhost:8080/actuator/health
```

Response:

```
{
  "status": "UP",
  "components": {
    "db": {
      "status": "UP"
    },
    "kafka": {
      "status": "UP"
    }
  }
}
```

Custom Health Endpoints

- /actuator/health : Overall health
- /actuator/health/db : Database health
- /actuator/health/kafka : Kafka health

Logging Configuration

Logback Configuration

Create logback-spring.xml :

```

<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <springProfile name="dev">
        <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
            <encoder>
                <pattern>%d{yyyy-MM-dd HH:mm:ss} - %msg%n</pattern>
            </encoder>
        </appender>
        <root level="INFO">
            <appender-ref ref="CONSOLE" />
        </root>
    </springProfile>

    <springProfile name="prod">
        <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
            <file>logs/application.log</file>
            <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
                <fileNamePattern>logs/application-%d{yyyy-MM-dd}.log</fileNamePattern>
                <maxHistory>30</maxHistory>
            </rollingPolicy>
            <encoder>
                <pattern>%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n</pattern>
            </encoder>
        </appender>
        <root level="INFO">
            <appender-ref ref="FILE" />
        </root>
    </springProfile>
</configuration>

```

Troubleshooting

Common Issues

Application won't start:

- Check Java version (must be 17+)
- Verify database connection
- Check port 8080 is available
- Review application logs

Database connection errors:

- Verify MySQL is running
- Check database credentials
- Ensure database exists
- Check network connectivity

Kafka connection errors:

- Verify Kafka is running
- Check Kafka broker address
- Verify topics exist
- Check network connectivity

Email sending fails:

- Verify SMTP credentials
- Check firewall rules
- Verify email server accessibility
- Check application logs

Maintenance

Regular Tasks

- **Daily:** Monitor application logs
- **Weekly:** Review error logs
- **Monthly:** Database optimization
- **Quarterly:** Security updates
- **Annually:** Major version updates

Update Procedure

1. Backup database
2. Stop application
3. Deploy new version
4. Run database migrations
5. Start application
6. Verify health checks
7. Monitor for issues

Support

For deployment support:

- Email: devops@vidyarthibookdepot.com
- Documentation: <https://docs.vidyarthibookdepot.com>
- Issue Tracker: <https://github.com/vidyarthibookdepot/issues>